

Unit 08: Virtual Functions

CONTENTS

Objectives

Introduction

8.1 Virtual functions

8.2 Rules for Virtual Functions

8.3 Late Binding

8.4 Early Bindings

8.5 Difference between Early Binding and Late Binding?

8.6 Pure Virtual Function

8.7 Virtual Function vs. Pure Virtual Function

Summary

Keywords

Answers for Self Assessment

Review Questions

Further Readings

Objectives

After studying this unit, you will be able to:

- Understand concept of binding in C++
- Understand early binding and late binding
- Demonstrate the virtual functions
- Recognize the pure virtual function

Introduction

At compile time, C++ defaults to matching a function call with the correct function definition. This is referred to as static binding. Dynamic binding allows you to tell the compiler to match a function call with the correct function definition at runtime. If you use the keyword `virtual` to declare a function, the compiler will use dynamic binding for that function.

8.1 Virtual functions

In C++ polymorphism refers to the property by which objects belonging to different classes are able to respond to the same message, but in different forms. An essential requirement of polymorphism is therefore the ability to refer to objects without any regard to their classes. This necessitates the use of a single pointer variable to refer to the objects of different classes. Here, we use the pointer to base class to refer to all the derived objects. But, we just discovered that a base pointer, even when it is made to contain the address of a derived class, always executes the function in the base class. The compiler simply ignores the contents of the pointer and chooses the member function that matches the type of the pointer. How do we then achieve polymorphism? It is achieved using what is known as 'virtual' functions.

When we use the same function name in both the base and derived classes, the function in base class is declared as virtual using the keyword `virtual` preceding its normal declaration. When a function is made virtual, C++ determines which function to use at run time based on the type of

object pointed to by the base pointer, rather than the type of the pointer. Thus, by making the base pointer to point to different objects, we can execute different versions of the virtual function.

**Notes**

- A virtual function is a member function which is declared within a base class and is re-defined (Overridden) by a derived class.
- Virtual functions ensure that the correct function is called for an object, regardless of the type of reference (or pointer) used for function call.
- Virtual functions mainly used to achieve run time polymorphism.
- Functions are declared with a virtual keyword in base class.

8.2 Rules for Virtual Functions

When virtual functions are created for implementing late binding, we should observe some basic rules that satisfy the compiler requirements:-

- The virtual functions must be members of some class.
- They cannot be static members.
- They are accessed by using object pointers.
- A virtual function can be a friend of another class.
- A virtual function in a base class must be defined, even though it may not be used.

**Lab Exercise**

```
//Program
#include <iostream>
using namespace std;
class Base {
public:
    virtual void print() {
        cout << "Base Function" << endl;
    }
};
class Derived : public Base {
public:
    void print() {
        cout << "Derived Function" << endl;
    }
};
int main() {
    Derived d1;
    Base* bptr = &d1;
    bptr->print();
    return 0;
}
```

Output

```
Derived Function
Process returned 0 (0x0)   execution time : 0.013 s
Press any key to continue.
```

8.3 Late Binding

Late binding refers to the act of selecting functions while the programme is running. Although late binding adds to the overhead, it gives you more power and flexibility. Due to the fact that late binding is implemented using virtual functions, we must specify a class object as either a pointer to a class or a reference to a class.



Did you know?

What is binding?

Binding refers to the process that is used to convert identifiers (such as variable and function names) into machine language addresses. Although binding is used for both variables and functions, but we have to focus on function binding.



For example the following shows how a late binding or run time binding can be carried out with the help of a virtual function.

```
class base {
private:
int x;
float y;
public:
virtual void display ();
int sum ();
};
class derivedD : public baseA
{
private:
int x;
float y;
public:
void display (); //virtual
int sum ();
};
void main ()
{
baseA *ptr;
```

```

derivedD objd;
ptr = &objd;
Other Program statements
ptr->display (); //run time binding
ptr->sum (); //compile time binding
}

```

Note that the keyword `virtual` is followed by the return type of a member function if a run time is to be bound. Otherwise, the compile time binding will be effected as usual. In the above program segment, only the `display ()` function has been declared as `virtual` in the base class, whereas the `sum ()` is non-virtual. Even though the message is given from the pointer of the base class to the objects of the derived class, it will not access the `sum ()` function of the derived class as it has been declared as non-virtual. The `sum ()` function compiles only the static binding.

The following program demonstrates the run time binding of the member functions of a class. The same message is given to access the derived class member functions from the array of pointers. As function are declared as `virtual`, the C++ compiler invokes the dynamic binding.

```

#include <iostream.h>
#include <conio.h>
class baseA {
public:
virtual void display () {
cout<< "One \n";
}
};
class derivedB : public baseA
{
public:
virtual void display () {
cout<< "Two\n"; }
};
class derivedC: public derivedB
{
public:
virtual void display () {
cout<< "Three \n"; }
};
void main () {
//define three objects
baseA obja;
derivedB objb;
derivedC objc;
base A *ptr [3]; //define an array of pointers to baseA
ptr [0] = &obja;
ptr [1] = &objb;

```

```
ptr [2] = &objc;
for (int i = 0; i <=2; i ++ )
ptr [i]->display (); //same message for all objects
getche ();
}
```

Output

One

Two

Three

The following programme demonstrates the static binding of a class's member functions. There are two classes in the programme: student and academic. The term "class academic" comes from the term "class student." Both classes have the member functions getdata and display specified. *obj is a variable for the class student, whose address is recorded in the class academic's object.

```
# include <iostream.h>
```

```
Using namespace std;
```

```
class student {
```

```
private:
```

```
int rollno;
```

```
char name [20];
```

```
public:
```

```
void getdata ();
```

```
void display ();
```

```
};
```

```
class academic: public student {
```

```
private:
```

```
char stream;
```

```
public:
```

```
void getdata ();
```

```
void display ();
```

```
};
```

```
void student:: getdata ()
```

```
{
```

```
cout<< "enterrollno\n";
```

```
cin>> rollno;
```

```
cout <<"enter name \n";
```

```
cin > > name;
```

```
}
```

```
void student:: display ()
```

```
{
```

```
cout<< "the student's roll number is "<<rollno<<"and name is"
```

```
<<name <<"Stream " <<stream;
```

```

cout<< endl;
}
void academic :: getdata ()
{
cout<< "enter stream of a student? \n";
cin >>stream;
}
void academic :: display () {
cout<< "students stream \n";
cout <<stream<< endl;
}
void main ()
{
student *ptr;
academic obj;
ptr=&obj;
ptr->getdata ();
ptr->display ();
}

```

Output

enter rollno

1

enter name

ajay

enter stream of a student?

IT

the student's roll number is 1 and name is ajay and Stream IT



Did you know?

What is dynamic binding?

You can specify the compiler match a function call with the correct function definition at run time; this is called dynamic binding.



Lab Exercise

```

#include <iostream>
using namespace std;
class Base
{
public:
virtual void print()

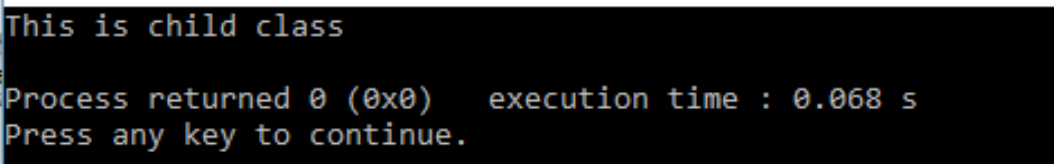
```

```

{
cout << "This is parent class" << endl;
}
};
class Derived : public Base
{
public:
void print()
{
cout << "This is child class" << endl;
}
};
int main()
{
Base *bptr;
Derived d;
bptr= &d;
bptr-> print();
return 0;
}

```

Output



```

This is child class
Process returned 0 (0x0) execution time : 0.068 s
Press any key to continue.

```

8.4 Early Bindings

- Most of the function calls the compiler encounters will be direct function calls. A direct function call is a statement that directly calls a function.
- It is also known as Static Binding or Compile-time Binding.
- Direct function calls can be resolved using a process known as early binding.
- Early binding (also called static binding) means the compiler is able to directly associate the identifier name (such as a function or variable name) with a machine address.
- All functions have a unique machine address. So when the compiler encounters a function call, it replaces the function call with a machine language instruction that tells the CPU to jump to the address of the function.



Example

// Program

```
#include <iostream>
```

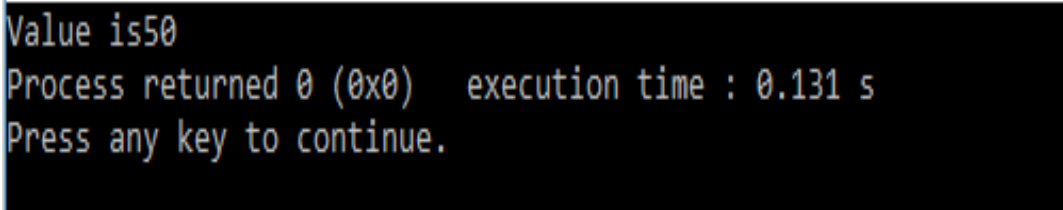
Object Oriented Programming Using C++

```
using namespace std;

void displayValue(int val){
    cout<<"Value is" << val;
}

int main(){
    displayValue(50); // Direct Function call
    return 0;
}
```

Output



```
Value is50
Process returned 0 (0x0)  execution time : 0.131 s
Press any key to continue.
```

8.5 Difference between Early Binding and Late Binding?

Let's see the difference according to multiple parameters between bindings

Early Binding vs Late Binding	
The process of using the class information to resolve method calling that occurs at compile time is called Early Binding.	The process of using the object to resolve method calling that occurs at run time is called the Late Binding.
Time of Binding	
Early Binding happens at compile time.	Late Binding happens at run time.
Functionality	
Early Binding uses the class information to resolve method calling.	Late Binding uses the object to resolve method calling.
Synonyms	
Early Binding is also known as static binding..	Late Binding is also known as dynamic binding.
Occurrence	
Overloading methods are bonded using early binding.	Overridden methods are bonded using late binding.
Execution Speed	
Execution speed is faster in early binding.	Execution speed is lower in late binding.

**Task**

- Explain the difference between early binding and late binding with suitable programming examples using C++.

8.6 Pure Virtual Function

It is normal practice to declare a function virtual inside the base class and redefine it in the derived classes. The function inside the base class is seldom used for performing any task. It only serves as a placeholder. For example, we have not defined any object of class media and therefore the function display() in the base class has been defined 'empty'. Such functions are called "do-nothing" functions.

A "do nothing function" may be defined as follows:

```
virtual void display()=0;
```

Such functions are called pure virtual functions. A pure virtual function is a function declared in a base class that has no definition relative to the base class. In such cases, the compiler requires each derived class to either define the function or re-declare it as a pure virtual function. Remember that a class containing pure virtual functions cannot be used to declare any objects of its own. As stated earlier, such classes are called abstract base classes. The main objective of an abstract base class is to provide some traits to the derived classes and to create a base pointer required for achieving run time polymorphism.

**Lab Exercise**

```
#include <iostream>
using namespace std;
class A
{
public:
    virtual void test() = 0;
};
class B : public A
{
public:
    void test()
    {
        cout << " Hello ! Pure Virtual function " << endl;
    }
};
int main(void)
{
    B obj;
    obj.test();
    return 0;
}
Output
```

```
Hello ! Pure Virtual function
Process returned 0 (0x0)   execution time : 0.102 s
Press any key to continue.
```

8.7 Virtual Function vs. Pure Virtual Function

Virtual function	Pure virtual function
A virtual function is a member function of base class which can be redefined by derived class.	A pure virtual function is a member function of base class whose only declaration is provided in base class and should be defined in derived class otherwise derived class also becomes abstract.
Classes having virtual functions are not abstract.	Base class containing pure virtual function becomes abstract.
Definition is given in base class.	No definition is given in base class.
Base class having virtual function can be instantiated i.e. its object can be made.	Base class having pure virtual function becomes abstract i.e. it cannot be instantiated.
If derived class do not redefine virtual function of base class, then it does not affect compilation.	If derived class do not redefine virtual function of base class, then no compilation error but derived class also becomes abstract just like the base class.
All derived class may or may not redefine virtual function of base class.	All derived class must redefine pure virtual function of base class otherwise derived class also becomes abstract just like base class.

Summary

- OOP is used commonly for software development. One major pillar of OOP is polymorphism. Early Binding and Late Binding are related to that. Early Binding occurs at compile time while Late Binding occurs at runtime. In method overloading, the bonding happens using the early binding. In method overriding, the bonding happens using the late binding. The difference between Early and Late Binding is that Early Binding uses the class information to resolve method calling while Late Binding uses the object to resolve method calling.
- When a function is made virtual, C++ determines which function to use at run time based on the type of object pointed to by the base pointer, rather than the type of the pointer. By making the base pointer to point to different objects, we can execute different versions of the virtual function.
- If a virtual function is defined in the base class, it need not be necessarily redefined in the derived class. In such cases, the respective calls will invoke the base class function.
- A virtual function, equated to zero is called a pure virtual function. It is a function declared in a base class that has no definition relative to the base class. A class containing such pure function is called an abstract class.
- Early binding refers to the events that occur at compile time while late binding means selecting function during the execution. The late binding is implemented through virtual function.

Keywords

Late Binding: Selecting functions during the execution. Though late binding requires some overhead it provides increased power and flexibility.

Virtual Function: Virtual functions, one of advanced features of OOP is one that does not really exist but it appears real in some parts of a program.

Compile time polymorphism: Compile-time polymorphism is achieved through method overloading.

Run time polymorphism: The runtime polymorphism can be achieved by method overriding.

Do nothing function: The Do-Nothing method is used to arrange that a call to a method or property should be ignored. When using Do-Nothing, all the logic inside the arranged method or property body is skipped and nothing happens when you call it.

Self Assessment

1. Which is the correct declaration of pure virtual function in C++
 - A. virtual void func = 0;
 - B. virtual void func() = 0;
 - C. virtual void func(){0};
 - D. void func() = 0;
2. In a class, pure virtual functions in C++ is used
 - A. To create an interface
 - B. To make a class abstract
 - C. To force derived class to implement the pure virtual function
 - D. All the above
3. Virtual functions mainly used to achieve run time polymorphism.
 - A. True
 - B. False
4. The prototype of virtual functions should be the same in the base as well as derived class.
 - A. True
 - B. False
5. If a is defined in the base class, it need not be necessarily redefined in the derived class.
 - A. member function
 - B. virtual function
 - C. static function
 - D. real function
6. is a function declared in a base class that has no definition relative to the base class.
 - A. member function
 - B. virtual function
 - C. pure virtual function
 - D. pure function
6. State, whether the following statements about virtual functions are True.

- i) The virtual function must be a member of some class
 - ii) virtual functions cannot be static members
 - iii) A virtual function cannot be a friend of another class.
- A. i and ii only
B. ii and iii only
C. i and iii only
D. All i, ii and iii
7. State whether the following statements about virtual functions are True or False.
- i) A virtual function, equated to zero is called pure virtual function.
 - ii) A class containing pure virtual function is called an abstract class
- A. True, True
B. True, False
C. False, True
D. False, False
8. In compile-time polymorphism, a compiler is able to select the appropriate function for a particular call at the compile time itself, which is known as
- A. early binding
B. static binding
C. static linking
D. All of the above
9. binding means that an object is bound to its function call at compile time.
- A. late
B. static
C. dynamic
D. fixed
10. Which of the following concepts means waiting until runtime to determine which function to call?
- A. Data hiding
B. Dynamic loading
C. Dynamic binding
D. Data casting
11. C++ supports run time polymorphism with the help of virtual functions, which is called binding.
- A. dynamic
B. run time
C. early binding
D. static
12. Static binding is same as

- A. Run Time Binding
- B. Compile Time Binding
- C. All of above
- D. None of above

13. Use of virtual functions implies

- A. overloading
- B. overriding
- C. static binding
- D. dynamic binding

14. What would be the output of following code?

```
#include <iostream>

using namespace std;

int Add(int x,int y){
    return x+y;
}

int main(){
    int (*fptr)(int,int)=Add;
    cout<<fptr(10,40);
    return 0;
}
```

- A. 10
- B. 40
- C. 50
- D. Compile time error

Answers for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. B | 2. D | 3. A | 4. A | 5. B |
| 6. C | 7. A | 8. A | 9. D | 10. B |
| 11. C | 12. A | 13. B | 14. D | 15. C |

Review Questions

1. How can C++ achieve dynamic binding? Explain with suitable example.
2. What do you mean by binding in C++?
3. Differentiate between early binding and late binding.
4. Write a program using C++ that demonstrate working of dynamic binding.
5. Differentiate between virtual function and pure virtual function.
6. Pure virtual functions force the programmer to redefine the virtual function inside the derived class. Comment the statement.
7. Write a program using C++ that demonstrate working of pure virtual function.

8. Write a program using C++ that demonstrate working of static binding.
9. When do we make a virtual function “pure”? What are the implications of making a function a pure virtual function?
10. Why do we need virtual functions?



Further Readings

E Balagurusamy; Object-oriented Programming with C++; Tata Mc Graw-Hill.

Herbert Schildt; The Complete Reference C++; Tata Mc Graw Hill.

Robert Lafore; Object-oriented Programming in Turbo C++; Galgotia.



Web Links

<http://www.learncpp.com/cpp-tutorial/117-multiple-inheritance/>

<https://www.codecademy.com/learn/learn-c-plus-plus>

http://www.artima.com/cppsource/pure_virtual.html

<http://publib.boulder.ibm.com/infocenter/comphelp/v8v101/index.jsp?topic=%2Fcom.ibm.xlcpp8a.doc%2Flanguage%2Fref%2Fcplr142.htm>